

```
Create database SalesTable;
```

```
use SalesTable;
```

```
CREATE TABLE Customers (  
    customerid INT PRIMARY KEY,  
    customername VARCHAR(50),  
    city VARCHAR(50)  
);
```

```
INSERT INTO Customers (CustomerID, CustomerName, City) VALUES  
(1, 'Customer_1', 'New York'),  
(2, 'Customer_2', 'New York'),  
(3, 'Customer_3', 'New York'),  
(4, 'Customer_4', 'Chicago'),  
(5, 'Customer_5', 'Chicago'),  
(6, 'Customer_6', 'New York'),  
(7, 'Customer_7', 'New York'),  
(8, 'Customer_8', 'New York'),  
(9, 'Customer_9', 'New York'),  
(10, 'Customer_10', 'Chicago'),  
(11, 'Customer_11', 'Houston'),  
(12, 'Customer_12', 'Phoenix'),  
(13, 'Customer_13', 'Houston'),  
(14, 'Customer_14', 'Chicago'),  
(15, 'Customer_15', 'Chicago'),  
(16, 'Customer_16', 'Phoenix'),  
(17, 'Customer_17', 'Houston'),  
(18, 'Customer_18', 'Chicago'),  
(19, 'Customer_19', 'Los Angeles'),  
(20, 'Customer_20', 'Phoenix'),  
(21, 'Customer_21', 'Chicago'),  
(22, 'Customer_22', 'Los Angeles'),  
(23, 'Customer_23', 'Phoenix'),  
(24, 'Customer_24', 'New York'),  
(25, 'Customer_25', 'Houston');
```

```
select * from customers;
```

```
CREATE TABLE Products (  
    productid INT PRIMARY KEY,  
    productname VARCHAR(50),  
    price DECIMAL(10,2)
```

);

INSERT INTO Products (ProductID, ProductName, Price) VALUES

(1, 'Product\_1', 29.75),  
(2, 'Product\_2', 12.86),  
(3, 'Product\_3', 19.00),  
(4, 'Product\_4', 68.60),  
(5, 'Product\_5', 77.10),  
(6, 'Product\_6', 29.08),  
(7, 'Product\_7', 25.74),  
(8, 'Product\_8', 37.62),  
(9, 'Product\_9', 59.06),  
(10, 'Product\_10', 98.14),  
(11, 'Product\_11', 38.13),  
(12, 'Product\_12', 90.97),  
(13, 'Product\_13', 17.88),  
(14, 'Product\_14', 70.26),  
(15, 'Product\_15', 24.75),  
(16, 'Product\_16', 85.02),  
(17, 'Product\_17', 28.75),  
(18, 'Product\_18', 74.40),  
(19, 'Product\_19', 71.02),  
(20, 'Product\_20', 23.62),  
(21, 'Product\_21', 29.79),  
(22, 'Product\_22', 31.25),  
(23, 'Product\_23', 96.69),  
(24, 'Product\_24', 77.87),  
(25, 'Product\_25', 34.99);

CREATE TABLE Orders (

    orderid INT PRIMARY KEY,

    customerid INT,

    productid INT,

    orderdate DATE,

    quantity INT,

    FOREIGN KEY (customerid) REFERENCES Customers(customerid),

    FOREIGN KEY (productid) REFERENCES Products(productid)

);

INSERT INTO Orders (OrderID, CustomerID, ProductID, OrderDate, Quantity) VALUES

(1, 12, 1, '2025-09-30', 5),  
(2, 7, 2, '2025-03-29', 4),  
(3, 6, 13, '2025-01-24', 5),

```
(4, 3, 5, '2025-05-10', 3),
(5, 17, 9, '2025-08-15', 1),
(6, 19, 17, '2024-11-13', 4),
(7, 16, 14, '2025-09-19', 4),
(8, 21, 16, '2024-11-10', 4),
(9, 18, 5, '2025-02-23', 2),
(10, 25, 20, '2025-02-20', 5),
(11, 18, 20, '2025-01-02', 4),
(12, 25, 1, '2025-06-17', 5),
(13, 18, 8, '2025-05-12', 2),
(14, 12, 16, '2025-10-31', 3),
(15, 20, 4, '2024-11-16', 3),
(16, 9, 16, '2025-02-04', 1),
(17, 6, 18, '2024-11-11', 1),
(18, 18, 18, '2025-03-24', 3),
(19, 14, 13, '2025-08-28', 1),
(20, 21, 14, '2024-12-28', 4),
(21, 1, 9, '2025-08-12', 4),
(22, 2, 12, '2024-12-24', 5),
(23, 17, 14, '2025-05-22', 2),
(24, 25, 25, '2024-12-28', 2),
(25, 5, 17, '2025-10-03', 1);
```

```
select * from orders;
```

-- List all orders with customer names and product names.

```
select orderid , customername , productname
from orders
INNER JOIN Customers on Customers.customerid = orders.customerid
INNER JOIN Products on Products.productid = orders.productid;
```

-- Find all customers who placed an order for "Product\_14".

```
select customername , productname
from orders
INNER JOIN customers on customers.customerid = orders.customerid
INNER JOIN products on products.productid = orders.productid
where productname = 'Product_14';
```

-- Show total quantity ordered per customer.

```
select customername , sum(quantity) as Total_Orders
from orders
INNER JOIN customers on customers.customerid = orders.customerid
INNER JOIN products on products.productid = orders.productid
group by customername;
```

-- List customers who have never placed an order.

```
select customername , orderid from customers
LEFT JOIN orders
on customers.customerid = orders.customerid
where orderid is NULL;
```

-- Find products that have never been ordered.

```
select productname , orderid from orders
RIGHT JOIN products
ON products.productid = orders.productid
where orderid is NULL;
```

-- Show total revenue per product (Price × Quantity).

```
select productname , sum((price * quantity)) as Revenue_Per_Product from products
INNER JOIN orders
on products.productid = orders.productid
group by productname;
```

-- List orders placed in the last 90 days with customer and product details.

```
select productname , orderid from orders
INNER JOIN products
ON products.productid = orders.productid
where datediff(now() , orderdate) <= 90;
```

-- Find the most frequently ordered product.

```
select productname , count(products.productid) as TotalOrder
from products INNER JOIN orders
```

```
on products.productid = orders.productid
group by productname
order by TotalOrder DESC
LIMIT 1;
```

-- List cities with total number of orders placed.

```
select city , count(orders.customerid) TotalOrders from customers
INNER JOIN orders
on customers.customerid = orders.customerid
group by city;
```

-- Find customers who ordered products priced above 50.

```
select customername , productname , orderid , price from orders
INNER JOIN customers on orders.customerid = customers.customerid
INNER JOIN products on orders.productid = products.productid
where price > 50;
```

-- Show products ordered by customers from "Chicago".

```
select customername , productname , orderid , city from orders
INNER JOIN customers on customers.customerid = orders.customerid
INNER JOIN products on products.productid = orders.productid
where city = 'Chicago';
```

-- List customers who ordered the same product more than once.

```
select customername , productname , count(productname) as TotalOrder from orders
INNER JOIN customers on customers.customerid = orders.customerid
INNER JOIN products on products.productid = orders.productid
group by customername , productname
having TotalOrder > 1;
```

-- Show the latest order date for each customer.

```
select customername , MAX(orderdate) as LatestOrder from orders
INNER JOIN customers on customers.customerid = orders.customerid
```

group by customername;

-- find the top 3 customers by total spending.

```
select customername , SUM(price * quantity) as TotalSpend from orders
INNER JOIN customers on customers.customerid = orders.customerid
INNER JOIN products on products.productid = orders.productid
group by customername
order by TotalSpend DESC
LIMIT 3;
```

-- Show average quantity ordered per product.

```
select productname , avg(quantity) as Average_Quantity from orders
INNER JOIN products on products.productid = orders.productid
group by productname;
```

-- List all orders with product price and total cost.

```
select orderid , productname , price , quantity , (price * quantity) as TotalCost from orders
INNER JOIN products on products.productid = orders.productid;
```

-- (sub queries)

-- Find customers who placed the highest quantity order.

```
select distinct customername , quantity from orders
INNER JOIN customers On customers.customerid = orders.customerid
where quantity = (select max(quantity) from orders);
```

-- List products with price above the average product price.

```
select * from products where price > (select avg(price) from products);
```

-- Find the most expensive product ordered.

```
select * from products where price = (select max(price) from products);
```

-- List customers who ordered the cheapest product

```
select customername , productname , price from orders
INNER JOIN customers on customers.customerid = orders.customerid
INNER JOIN products on products.productid = orders.productid
where price = (
    select min(price) from products
```

```
);
```

```
-- queries i failed do it by myself
```

```
-- List products ordered by more than 5 different customers.
```

```
SELECT
    products.productname,
    COUNT(DISTINCT orders.customerid) AS Total_Customers
FROM orders
INNER JOIN products
    ON products.productid = orders.productid
GROUP BY products.productname
HAVING COUNT(DISTINCT orders.customerid) > 5;
```

```
-- Show customers who ordered more than 3 different products.
```

```
select customername , count( distinct orders.productid) as TotalOrders
from orders
INNER JOIN customers
on customers.customerid = orders.customerid
group by customername
having count( distinct orders.productid) > 3;
```

```
-- Find customers who ordered both "Product_1" and "Product_25".
```

```
select customername from orders
INNER JOIN customers on customers.customerid = orders.customerid
INNER JOIN products on products.productid = orders.productid
where products.productname in ('Product_1' , 'Product_25')
group by customername
HAVING COUNT(DISTINCT products.productname) = 2;
```

```
-- Show customers whose total spending is above the average spending.
```

```
select customername , sum(price * quantity) as TotalSpending from orders
INNER JOIN customers on customers.customerid = orders.customerid
INNER JOIN products on products.productid = orders.productid
group by customername;
```

```
-- Remaining
```

```
-- Show customers who ordered products from multiple cities (if product location is added later).
```